



Benchmarking Credit Card Fraud Detection Models: A Comprehensive Evaluation Across Diverse Datasets

Deepankur Singh¹ · Simranjit Singh¹ · Neeraj Kumar¹

Received: 22 January 2025 / Accepted: 4 December 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

Credit card fraud continues to present substantial economic challenges, driven by the dynamic nature of fraudulent behavior and the inherent class imbalance in transactional data. In this study, a comprehensive comparison of fifteen machine learning algorithms, including classical classifiers (Logistic Regression, Support Vector Machine, and Ridge Classifier), discriminant analysis techniques (Linear and Quadratic Discriminant Analysis), distance-based models (K-Nearest Neighbours), probabilistic approaches (Naive Bayes), a single decision tree, ensemble learning methods (Random Forests, Extremely Randomised Trees, Extreme Gradient Boosting, Categorical Boosting, and Light Gradient Boosting Machine), and a deep learning model designed for tabular data (TabNet) is conducted. Experiments were conducted using three datasets: the 2013 European credit card transaction dataset, a synthetic dataset from 2019 and a real-world dataset collected in 2023. To address the severe class imbalance, the Synthetic Minority Oversampling Technique and Adaptive Synthetic Sampling is employed. The results indicate that ensemble methods, particularly Extreme Gradient Boosting and Categorical Boosting, achieve high accuracy and efficiency. Although TabNet delivers the best performance on the balanced 2023 dataset, its higher computational cost limits its practical deployment. Interpretability analysis using SHapley Additive exPlanations (SHAP) identifies transaction amount and features derived through Principal Component Analysis as key indicators of fraudulent activity. These insights support the development of interpretable, data-driven fraud detection models with real-world economic relevance. The study code is available at: <https://github.com/Deep8s/CCFD.git>

Keywords Benchmarking · Credit card fraud · Ensemble methods · Fraud detection · Machine learning models · SMOTE · SHAP

1 Introduction

A credit card is a small plastic card that allows payments to be performed at the click of a button. The credit card transactions are performed through a machine called a Point-of-Sale (POS) or Automated Teller Machine (ATM). The security of cards involves two aspects: physical protection and privacy. Globalisation has led to a rapid growth of e-commerce services, which in turn has increased online transactions. As a result, the usage of credit cards surged. As credit card usage increased, there was a notable growth in fraudulent activities. Fraud related to credit cards can be in many forms and the fraudsters make a wide range of strategies to commit crimes (Pandarinath et al., 2024). There are many types of credit card fraud, each of which exploits different types of vulnerabilities, as shown in Fig. 1. In the case of account takeover, fraudsters use personal data to take control of a user's account. A very common type of fraud in online transactions is Card-not-present (CNP), which does not require physical card verification. Application fraud refers to the practice of providing false or incorrect information to obtain credit card details. Fraudsters trick people by sending fake emails to obtain personal information from users. Phishing and skimming collect card data during legitimate transactions and store it for future use. Lost/stolen card fraud is the easiest way for criminals to obtain the user's card details through theft or loss.

Fraudsters create many fake and counterfeit cards by either modifying original cards or using specialised tools to make new ones. When there is collusion between the merchants and the fraudsters then there is Merchant-related frauds or crime such as Triangulation, a scam where scammer collect the data through the fake websites or fake online stores and use that information to make the legitimate transactions on the legitimate sites. Fraudsters create false websites and clone existing sites to gather information from users; these types of fraud are called Internet-related frauds.

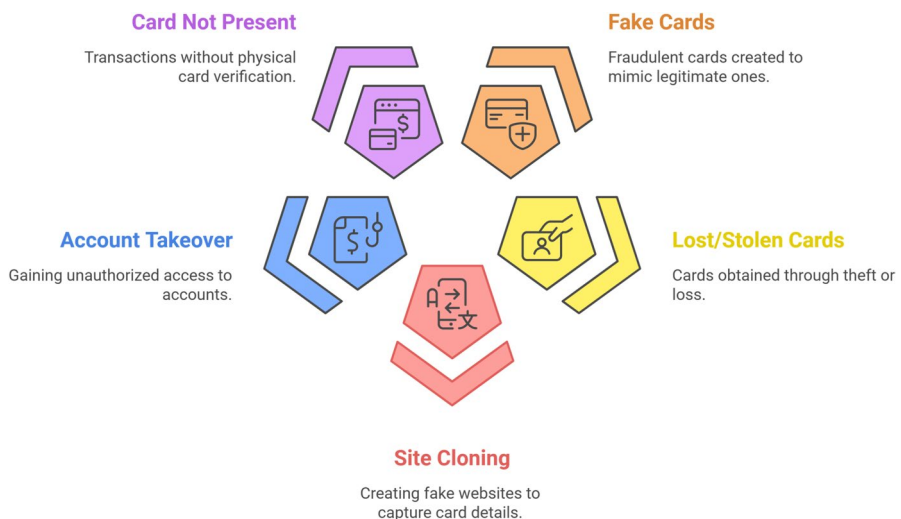


Fig. 1 Types of credit card fraud

The recent developments in digital payment systems have created new vulnerabilities to exploit. Fraudsters use various tactics and techniques with the changing security mechanism. According to Nilson report (2023), the card fraud exceeded \$32 billion in 2022 and it is expected to be increase to \$38.5 billion by 2027 worldwide (Nilson, 2023). Many financial institutions, consumers and the merchants are affected by this growing issue of the credit card fraud, so there is need for the efficient fraud detection systems that can minimize the financial damage.

As shown in Fig. 2, the global fraud rate and credit card fraud losses were \$18.11 billion in 2014, with a predicted increase to \$43.47 billion by 2028. Although we have advanced detection systems, fraudsters continue to adapt their criminal behaviour, and the ongoing increase in online or digital transactions contributes to a rise in losses. Unauthorized use of credit cards has increased in recent years due to the rise in online shopping and e-commerce.

Credit card fraud has become a significant challenge in the modern financial sector. The rise of e-commerce, digital payments, and online transactions has increased risks for consumers and financial institutions alike. Each year, fraud results in billions of dollars in financial losses, which eat away the consumer trust and threaten the stability of financial ecosystem. Historically, fraud detection systems relied on predefined rules and fixed heuristics to detect suspicious activities. However, as fraudsters adopt more sophisticated techniques, these traditional methods often fall short, leading to false positives which in turns increase the cost. Machine learning has emerged as a powerful alternative, offering the ability to detect patterns, anticipate fraudulent activities, and adapt to evolving schemes in real time (Table 1).

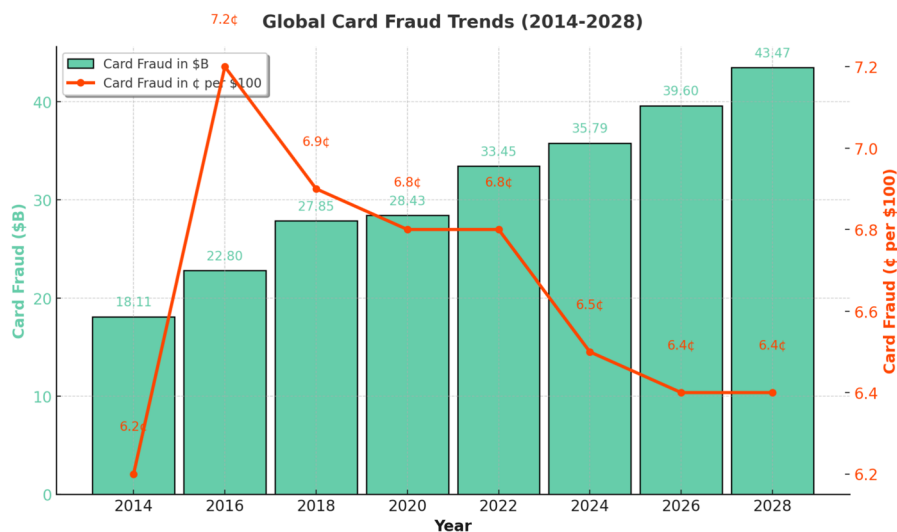


Fig. 2 Nilson Report 2023 (Nilson, 2023)

Table 1 Types of credit card fraud with descriptions and sample scenarios

Fraud Type	Description	Sample Scenarios
Card-Present Fraud	Unauthorized use of stolen or cloned physical cards.	Using skimming devices on ATMs or POS terminals to duplicate card data.
Card-Not-Present Fraud	Online misuse of stolen card details.	Making unauthorized purchases on e-commerce platforms using stolen credentials.
Phishing Attacks	Deceptive methods to trick users into revealing sensitive information.	Fraudulent emails or fake websites impersonating banks to steal cardholder data.
Synthetic Identity Fraud	Combining real and fake information to create new identities.	Opening credit accounts or obtaining loans using false identities.
Account Takeover	Unauthorized access to user accounts through compromised credentials.	Gaining control of accounts to initiate fraudulent transactions.
Merchant Fraud	Fraudulent activities by dishonest merchants.	Creating fake transactions or issuing fraudulent refunds to steal money.
Friendly Fraud	Legitimate users dispute valid transactions for personal gain.	Claiming non-receipt of goods or services to receive refunds.
Chargeback Fraud	Reversal of legitimate payments by falsely claiming fraud or non-delivery.	Disputing transactions after receiving goods or services.
Interception Fraud	Stealing cardholder details during the transaction process.	Exploiting unsecured payment systems to intercept data in transit.
Card Cloning	Copying data from a card's magnetic strip or chip for misuse.	Using skimmers to copy card details for unauthorized transactions.
Lost or Stolen Card Fraud	Transactions made with a card reported as lost or stolen.	Using a stolen card for purchases before it is reported.
Application Fraud	Opening a credit card account using fake or stolen identity details.	Applying for a credit card with stolen personal information.
Mail Theft Fraud	Stealing credit cards or billing statements from a victim's mail.	Taking a new credit card from a mailbox for unauthorized use.
Identity Theft Fraud	Using another person's personal information to open credit accounts.	Applying for credit cards in the victim's name without their knowledge.

1.1 Motivation

The increase in online shopping, business dealings, and credit card transactions has led to new risks, resulting in increasingly complex credit card fraud. Motivations for our research are given below:

1. Traditional methods/approaches fail to detect the credit card fraud due to frequently changing strategies or patterns by the fraudsters. So a comprehensive analysis of different ML models is needed on credit card fraud datasets for evaluating the performance and model behavior so that an optimal approach can be identified.
2. There are many challenges, such as unbalanced datasets, changing patterns of the fraudsters, and minimizing errors in the detection systems.

This research aims to test and compare different machine learning models on different datasets to find out the most effective methods which work best for detecting real-world fraud.

1.2 Contributions

The main contribution of this study can be summarized as follows:

1. In this study, a comprehensive analysis of different ML models was carried out on ML models, including Random Forest, XGBoost, Extra Trees classifier, Cat-Boost etc and TabNet, a deep neural network, on three different datasets.
2. Employed three different datasets with different characteristics to check how the models work in different situations.
3. For data balancing, various methods have been explored, like SMOTE and ADASYN, which help to improve the model performance, especially in fraud detection.
4. Each model's evaluation was performed using many different performance metrics, such as accuracy, precision, recall, F1 score, MCC, Kappa, and training time (TT).

2 Related Work

The significant financial impact of credit card fraud on consumers and organisations has led to extensive research on its detection. In the past, early fraud detection efforts relied mostly on traditional methods such as statistical methods like regression and Z-scores, and they were used to detect unusual patterns by detecting the deviation from the normal patterns. But these methods struggled to handle the complex patterns or continuously changing fraud patterns.

A rule-based system is built on simple and easy-to-understand rules, and it is widely used. These systems were easy to understand and use but struggled when fraud patterns changed or where the number of transactions was higher.

Bolton and Hand (2002) were among the first to study the statistics for detecting fraud (Bolton & Hand, 2002). Later, Phua et al. (2010) used basic data mining techniques and showed that these methods could work well for finding the fraud in organised and complex data (Phua et al., 2010).

Worldwide, the banking sector loses more than \$32 billion every year because of credit card fraud. In order to minimize the losses the ML based techniques are employed (Nilson, 2023). Previous research has shown that both supervised and unsupervised machine learning (ML) models can effectively help in fraud detection. Simple models like logistic regression (LR) and decision trees (DT) are known for their simplicity, whereas ensemble methods such as random forest (RF) and gradient boosting improve the detection rate (Khalid et al., 2024a). Neural networks and Support Vector Machines (SVMs) can easily detect more complex fraud patterns but need more computational power (Patidar et al., 2011). However, most of the studies focus on the single dataset, but it makes it hard to know the model behaviour. This study aims to overcome that issue by testing and comparing models across multiple datasets. ML has changed the fraud detection by detecting fraud instantly. Traditional models like LR and DT became very popular because of their ease of understanding;

however, these models struggled sometimes with identifying more complex fraud patterns and handling unbalanced datasets (Randhawa et al., 2018).

Zareapoor and Shamsolmoali (2015) highlighted the use of ensemble learning techniques, such as bagging, to detect fraud in datasets where normal and fraudulent transactions are highly imbalanced. For example, PayPal uses the ensemble methods with advanced feature engineering methods to detect fraudulent activities. Research by Randhawa et al. (2018) found that the RF model can improve fraud detection performance, especially in recall, when compared to the simpler models. Dal Pozzolo et al. (2015) conducted an important study on unbalanced datasets in fraud detection to see how to deal with them. They tested different methods, such as undersampling and ML techniques, to make the fraud detection more effective. This study also addresses the practical issues, like the changing patterns of fraud over time, and also calls the problem ‘concept drift’, which helps in improving the fraud detection system.

To handle this problem, techniques like SMOTE, ADASYN, and cost-sensitive learning are being used by the researchers (Khalid et al., 2024a; Dal Pozzolo et al., 2015; Zhu et al., 2024; He et al., 2008). SMOTE was firstly introduced by Chawla et al. (2002), and companies like Stripe and Adyen have used it to spot the fraud. However, SMOTE does oversample the minority class and sometimes generate the fake transactions, which is unrealistic. This can make the model less accurate and untrustworthy during the time of detection.

Khalil et al. (2022) created a special dataset of 11 Egyptian insurance companies over 20 years (1999–2019), focusing on 22 key financial ratios. This study shows the advanced ML methods, especially the bagging and boosting methods, which performed better than the classical models like the SVM and RF. These advanced methods not only increase the performance but also reduce the errors. Chhabra et al. (2024) worked on the complex credit card fraud patterns in the datasets where the fraudulent cases are more than the normal cases. They created an ensemble model that combined three algorithms: RF, LR, and KNN. For data handling they use a mix of undersampling and oversampling techniques. They were able to identify more fraud cases and improve the overall accuracy of their fraud system.

Khalid et al. (2024a) suggested ensemble methods that combine models such as SVM, KNN, RF, bagging and boosting classifiers. These ensemble methods have shown better performance in complex relationships with the handling of unbalanced datasets more effectively when used with oversampling techniques.

The class imbalance problem in the dataset is a major challenge in fraud detection because fraudulent transactions are less than 1% of all transactions. To address the issues caused by the continuously changing fraud patterns and the unbalanced datasets, Awoyemi et al. (2017) evaluated three models: Naive Bayes (NB), KNN, and LR.

Zhu et al. (2024) proposed an approach that combines Neural Network (NN) with SMOTE to improve fraud detection performance. The research showed that this combination addresses the class imbalance problem and helps the model better identify rare fraud transactions. Khalil et al. (2024) focused on two main challenges in detecting the fraudulent insurance claims: missing data and a large imbalance between fraud and non-fraud transactions. To solve these issues, they used a mix of data imputation techniques like mean substitution, KNN and predictive modelling with oversampling methods such as SMOTE and ADASYN to make the dataset balanced. The

author's approach results in improvement, achieving up to a 15% increase in both recall and F1 score.

Khalid et al. (2024b) propose their work to make the fraud detection systems more dependable, especially when dealing with changing patterns (concept drift), imbalanced datasets and the demand for real-time decisions. They developed a comprehensive ensemble model that combined multiple models, including SVM, KNN, RF, bagging and boosting. To handle the class imbalance problem, they used a mix of undersampling and oversampling with SMOTE. Hafez et al. (2025) conducted a comprehensive review of AI-based fraud detection methods which examined over thirty studies published between 2019 and 2024. In the work the authors have taken ML, DL, and meta-heuristic optimisation while also pointing out the limitations with constantly changing fraudsters' behaviour.

In real life, many fraud detection systems use a mix of oversampling and anomaly detection to spot unusual activity. For example, American Express uses anomaly detection along with customer profiles to notice when someone's spending habits look suspicious, which helps them catch rare fraud cases more accurately. However, these types of systems struggle with issues like the false alarm, where legitimate transactions are incorrectly flagged as fraud. This can cause inconvenience to the customers because their real payment may get blocked.

Researchers normally assess how a fraud detection system works using common measures like accuracy, precision, recall, and F1-score (Kohavi, 1998; Powers, 2020). But these systems do not work well all the time, especially with imbalanced data. That's why cost-based measures, such as profit-focused evaluation, are important; they help us understand the real financial impact on the business when a fraud is missed or when a genuine transaction is wrongly marked or flagged (Dal Pozzolo et al., 2015). Most of the studies work only on the single dataset, so there is a demand for proper comparison across the diverse datasets of different types.

2.1 Gaps in Related Work

After studying the existing research. It is clear that several important areas haven't been explored yet. The gaps in the research discussed below highlight the promising directions for future studies that help in making improvements in the fraud detection.

- **High False Positive Rate:** Many fraud detection systems accidentally block real transactions, treating them as fraud cases. This not only frustrates customers but also leads to financial losses for businesses. The issue is especially common in systems that rely heavily on fixed, rule-based approaches.
- **Imbalanced Datasets:** Fraudulent transactions are much less in comparison to the normal ones. Because of the lower number of fraud transactions, the ML models struggle to find the frauds correctly unless a special method is used to balance the unbalanced datasets.

3 Data and Methodology

Ensemble methods that combine multiple models, such as Random Forest, Extra Trees, XGBoost, and LightGBM, have proven to work very well when dealing with datasets where fraud cases are much rarer than normal ones. Gradient-based techniques such as Gradient Boosting Classifier and CatBoost further improve the accuracy of predictions. Furthermore, statistical models like logistic regression and ridge regression are appreciated for their clarity and interpretability, while more sophisticated methods such as linear discriminant analysis and quadratic discriminant analysis adeptly manage intricate and multivariate datasets.

3.1 Dataset Specifications

3.1.1 European Cardholders Dataset

The dataset presented by Dal Pozzolo et al. (2015) includes credit card transactions from European cardholders over a two-day span in September 2013. In total, there are 284 807 transactions, with only 492 labelled as fraudulent, resulting in a fraud rate of about 0.172% and underscoring the severe class imbalance. The dataset is available at OpenML Contributors (2025).

Every feature in the dataset is numerical and was obtained through Principal Component Analysis (PCA) to preserve the privacy of the original attributes. These transformed variables are named $V_1, V_2, \dots, V_{28}$. Additionally, the dataset contains two original features that were not subjected to PCA:

- **Time:** The elapsed time (in seconds) from the first recorded transaction to the current transaction.
- **Amount:** The monetary value of the transaction, which may be useful for cost-sensitive learning techniques.
- **Class (Target):** A binary indicator denoting whether the transaction is fraudulent (1) or legitimate (0).

Due to privacy rules, the PCA-based features V_1 – V_{28} and other background details have not been shared publicly. This dataset is often used to test and compare fraud detection models, especially in situations where fraud cases are very rare and the input data has been anonymised.

3.1.2 Credit Card Transactions Fraud Detection Dataset 2019

The synthetic credit card transaction dataset used in this study covers the period from January 1, 2019, to December 31, 2020. It was created with the Sparkov Data Generation tool developed by Brandon Harris (Kartik2112, 2012). The dataset includes about 1 300 000 transactions made by 1 000 different customers and 800 merchants. To make the data realistic, customer and merchant profiles were defined using preset demographic and spending patterns – for example, customers aged 25–50 living in rural areas

or merchants selling electronics and clothing. The Python `faker` library was then used to generate fake but realistic transaction records that mimic real spending behaviour.

The consolidated dataset has 23 columns, which include:

- `trans_date_trans_time`: Timestamp for the date and time of each transaction.
- `cc_num`: Anonymised credit-card number.
- `merchant`, `category`: Merchant identifier and business category.
- `amt`: The amount involved in the transaction.
- Demographic and geographic attributes such as `gender`, `city`, `state`, `zip`, `city_pop`, `lat`, and `long`.
- `is_fraud`: Binary label indicating whether a transaction is fraud (1) or not fraud (0).

This dataset is easy to understand, made to mimic real-life transactions, and includes both the transaction information and extra related details. Because of this, it is very useful for building and testing machine learning models that detect credit card fraud (Kartik2112, 2012).

3.1.3 Credit Card Fraud Detection Dataset 2023

The dataset created by Nelgiriye withana (2023) contains over 550,000 anonymised credit card transactions made by European customers in 2023. It is mainly used for developing and testing computer models that can detect fraudulent transactions.

- **Features** **ID**: A unique ID number given to each transaction to identify it.
- **V_1 - V_{28}** : hidden details by PCA about a transaction due to privacy of customers.
- **Amount**: The amount of money involved in the transactions.
- **Class**: Binary label indicating whether a transaction is fraud (1) or not fraud (0).
- **Potential Applications** **Fraud Detection**: Using the provided data to train and test machine learning models so they can tell the difference between real and fraudulent transactions.
- **Merchant Category Analysis**: Looking for how different merchants are linked to the chances of the transaction being fraudulent.
- **Transaction Type Analysis**: Examining whether some types of transactions, like online purchases versus in-store purchases, are more likely to be fraudulent.

3.2 Model Selection

3.2.1 Ensemble Models

Ensemble methods combine multiple learning algorithms, often simple or weak models, to improve overall prediction accuracy.

XGBoost XGBoost is a very fast and powerful ensemble model. It is known for its features like regularization, parallel processing, and tree pruning. It is used mostly in the structured data like credit scoring, anomalies detection and ranking (Chen & Guestrin, 2016).

CatBoost CatBoost is a ML algorithm and it is a powerful gradient boosting library designed for categorical data without any need of heavy preprocessing and used in areas like e-commerce, finance, and recommendation systems (Prokhorenkova et al., 2018).

Gradient Boosting Classifier (GBC) GBC builds a series of model step by step where each model learns from mistakes of previous model. This helps in improving the accuracy of the model and works well when the data is unbalanced. It is widely used in risk assessment, healthcare decision etc (Friedman, 2001).

AdaBoost (ADA) AdaBoost easily improves the weak classifiers by mainly focusing more on the mistakes of previous rounds. It works well with binary classification tasks. It is mostly used in the tasks like facial recognition and fraud detection (Domingo et al., 2000; Freund & Schapire, 1997).

LightGBM LightGBM is designed to handle the large datasets efficiently and uses a histogram based method to make DT. It is faster than the XGBoost and uses less memory. It can also work with the categorical data. It is mostly used in tasks like develop trading strategies, optimizing energy usage (Manghi et al., 2018).

3.2.2 Linear Models

Linear models work on the idea that the input and output have a simple, straight-line relationship. In simple words, if one value increases or decreases, the result also changes in a direct and predictable way.

Logistic Regression (LR) LR is a ML model used for binary classification. It turns the input data into a probability to decide if something belongs to one group or the other. It is easy and run quickly and like data which can be separated in simple way like straight line (Cox, 1958).

Ridge Classifier (RIDGE) : This classifier is a type of a linear model which helps to reduces the overfitting by it stops the model from not learning too much noise from the dataset. For this it uses a small penalty to keep the model stable especially when there are multiple inputs and related to each other. It is commonly used in the text analysis and biological data research (Hoerl & Kennard, 1970).

3.2.3 Tree-Based Models

Tree based models are ML models which works and make prediction by splitting the data into small parts based on the certain rules.

Decision Tree (DT) DT is an ML model that splits the data into smaller parts to make final output. It does not need much data preparation as it is easy to understand and works well when patterns in data are not simple. Applications of DT are predicting credit score and in supply chain decisions (Breiman, 2007).

Random Forest (RF) RF is an ensemble method that uses multiple decision trees and combines them to achieve the better performance. It can be used for both classification and regression tasks and uses in the area of such as loan approval, medical diagnosis (Breiman, 2001).

Extra Trees (ET) ET is a faster version of the RF model it is similar to the random forest but it adds more randomness while making decision tree this makes it faster and accurate. It works well with the complex data or noisy data and can give more accurate and stable results without overfitting (Geurts et al., 2006).

3.2.4 Distance-Based Model's

These models make predictions by checking the similarity between one data point to others.

K-Nearest Neighbors (KNN) KNN is a method that classify the data by checking the closest data points around that point and used in the area such as recommendation system, recognizing images (Fix, 1985).

3.2.5 Naive Bayes Models

Naive Bayes models use probability to make predictions. It is based on Bayes' theorem and assumes that all the input features work independently of each other, even if that's not always true in real life.

Naive Bayes (NB) NB makes predictions by using probability. It is very fast and works well with small datasets and categorical data. It is used in applications like detecting spam emails, analyzing sentiments (Rish et al., 2001).

3.2.6 Support Vector Machines

Support Vector Machine (SVM) SVM is being used to transform the data into the higher dimensional data. It is widely used for the high dimensional data and helps in preventing the overfitting. It is used in bioinformatics, image classification and handwriting recognition (Cortes, 1995).

3.2.7 Dimensionality Reduction/Discriminant Analysis

These methods shrink down the information of data by reducing the number of features.

Linear Discriminant Analysis (LDA) LDA is an ML model which is used to classify the data. It tries to find the best straight line that separates two classes. In the case of fraud detection it can give fraud and non fraud cases separately. It reduces the dimensionality while keeping the information. It is commonly used in the area such as face recognition, bioinformatics, and customer segmentation (Fisher, 1936).

Quadratic Discriminant Analysis (QDA) QDA is very similar to the LDA, a supervised learning algorithm based on the Bayes theorem. It is an extended version of the LDA and is more flexible. For group separation LDA draws straight lines but the QDA draws curved lines. It is very good for large data and commonly used in the areas like Biometrics, pattern recognition and Medical diagnosis (Rao, 1948).

3.2.8 Deep Learning Model

Deep learning models are multilayered neural networks that learn from the complex and huge data like humans learn from experience.

TabNet It is a DL model designed by Google, specifically used for the tabular data such as spreadsheets, financial data, and medical records. It outperforms the traditional models (Arik & Pfister, 2021).

3.3 Preprocessing

In this study, three datasets are used; two datasets are unbalanced, and one dataset is balanced. Firstly, preprocessing is done so that the ML models could learn effectively. Average values are added in case of missing numbers and most common text value is added in the case of missing text. The experiment kept only up to 25 categories to avoid making the data too large. SMOTE and ADASYN are used to create extra samples to balance the dataset so that trained model would not show bias. In this study, 10-fold stratified cross validation to test the models fairly.

3.4 Training and Validation

The ML models reliable for the different datasets, the training and the validation procedures should be used in the very careful manner. First, the dataset is split into two parts 70% for training and 30% for testing of the model. The training sets ranged from 199,364 to 774,853 samples. Before training, data was cleaned by filling the missing values, handling the categorical values and scaling the numerical feature. All available CPU core were used to train the models. To help the models perform better and avoid overfitting, methods like hyperparameter tuning, stopping training early, and checkpoints were applied. A 10-fold stratified K-fold cross-validation was used to create balanced splits to test the model more thoroughly. For the final evaluation, a separate test set of about 85,443 to 170,589 samples was used with no preprocessing to test to avoid accidentally giving the model extra information. Model performance was measured using accuracy, AUC-ROC, precision, recall, and F1-score. Stratified sampling was applied to handle class imbalance and ensure fair results.

3.5 Performance Metrics

To provide a consistent basis for comparing all candidate models on the highly imbalanced credit-card fraud datasets, this study reports eight complementary evaluation metrics: accuracy (ACC), area under the ROC curve (AUC–ROC), precision (PPV), recall (sensitivity), F_1 -score (F_1), Cohen’s Kappa (κ), Matthews Correlation Coefficient (MCC), and normalized training time (TT).

3.5.1 Metric Definitions

Accuracy (ACC) Proportion of correctly classified transactions:

$$ACC = \frac{TP + TN}{TP + TN + FP + FN}. \quad (1)$$

AUC–ROC Area under the Receiver Operating Characteristic curve, quantifying the model’s ability to rank a fraudulent transaction above a legitimate one irrespective of decision threshold.

Precision (PPV) Fraction of transactions flagged as fraud that are actual fraud:

$$PPV = \frac{TP}{TP + FP}. \quad (2)$$

Recall (Sensitivity) Fraction of all true frauds correctly identified:

$$Recall = \frac{TP}{TP + FN}. \quad (3)$$

F_1 -Score (F_1) Harmonic mean of precision and recall:

$$F_1 = 2 \times \frac{PPV \times Recall}{PPV + Recall}. \quad (4)$$

Cohen’s Kappa (κ) Chance-adjusted agreement between predictions and ground truth:

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \quad (5)$$

where p_o is observed agreement and p_e is expected agreement by chance.

Matthews Correlation Coefficient (MCC) Balanced measure of binary classification quality:

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (6)$$

Training Time (TT) Wall-clock time (in seconds) required to train each model, normalized by the fastest model:

$$TT_i = \frac{\text{time}_i}{\min_j \text{time}_j}. \quad (7)$$

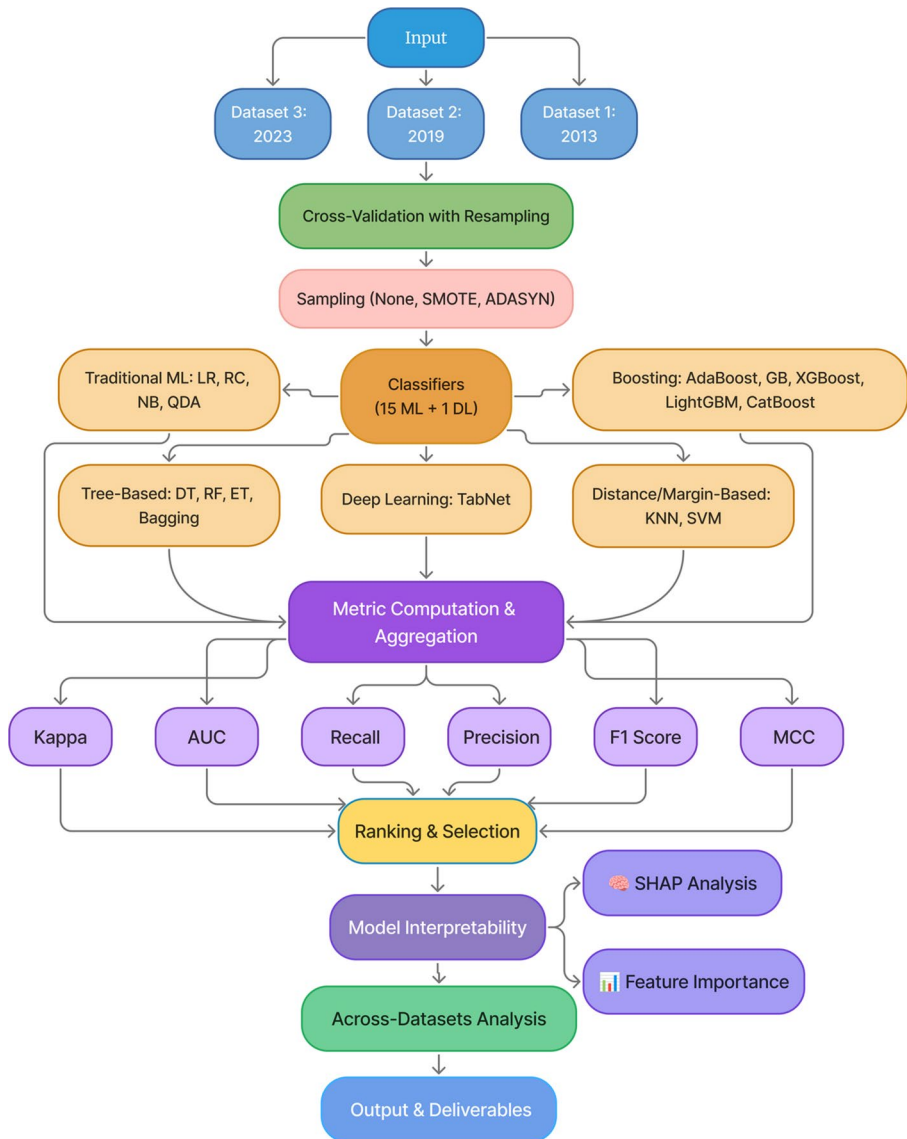


Fig. 3 Study framework

4 Experimental Results and Analysis

The experiments were performed on a 12th Generation Intel® Core™ i5-12500 processor, 16.0 GB of DDR5 RAM, and the Windows 11 software environment. We executed model development and analytical tasks using Jupyter Notebook (Kluyver et al., 2016; Pérez & Granger, 2007) in combination with Python 3.10 (Python Software Foundation, 2021; Van Rossum & Drake, 1995). The automation of machine learning tasks such as data preprocessing, model training, and evaluation—was carried out using PyCaret version 3.0.4 (Ali, 2020), providing an efficient framework for running and analyzing the experiments.

4.1 Algorithm

Algorithm 1 and Fig. 3 shows the general setup used in this study to detect credit card fraud. In this study three datasets (2013, 2019, 2023) were employed, along with fifteen ML models and one DL model, TabNet. For each data set, three different sampling strategies were used to see how well the models handle class imbalance (no oversampling, SMOTE, and ADASYN). The whole process follows the same steps for every dataset. In the first step, data cleaning and data preparation is performed. Then the stratified K-fold cross-validation is employed which is followed the model evaluation on different measures like accuracy, precision, recall, F1-score, Cohen's Kappa and MCC. Finally, the results were combined from all the experiments for the comparison of all three datasets.

Input: Three datasets: 2013, 2019, 2023
Output: Performance metrics and interpretability insights

```

1 foreach Dataset in {2013, 2019, 2023} do
2     Clean and prepare the data by filling in missing values, converting text or
       category data into numbers, and scaling numerical values so they're on a
       similar range.
3     Partition dataset using stratified K-fold cross-validation
4     foreach Sampling technique in {None, SMOTE, ADASYN} do
5         Apply sampling method on training partitions
6         foreach Model in 15 ML models and one DL model (TabNet) do
7             Train model on resampled training data
8             Evaluate on validation set using Accuracy, Precision, Recall,
              F1-score, Kappa, and MCC
9             Compute SHAP values for model interpretability
10            Analyze and visualize feature contributions to fraud prediction
11 Aggregate performance metrics across folds, sampling techniques, and datasets
12 Compare model performance and interpretability results
13 Generate final tables and visual summaries for analysis

```

Algorithm 1 Generalized Experimental Framework for Credit Card Fraud Detection

4.2 Results & Analysis

4.2.1 Performance on the 2013 Dataset

As shown in Table 2, when trained on the original, highly imbalanced data, tree-based ensembles, namely XGBoost, CatBoost, Random Forest, and Extra Trees, demonstrated the strongest discrimination, with AUC values ranging from 0.93 to 0.97, recall rates around 0.77–0.78, precision above 0.93, and F1 scores near 0.84–0.86. In contrast, TabNet failed entirely to flag any fraudulent transactions (Recall = 0), while simpler models lagged in either recall or precision. Among the ensembles, XGBoost excelled in efficiency, completing training in only 1.48 seconds, whereas Random Forest and Extra Trees required substantially more time.

After applying SMOTE or ADASYN to rebalance the classes, as shown in Tables 3 and 4, the top ensembles (Random Forest, Extra Trees, XGBoost, CatBoost, LightGBM, and Decision Tree) achieved perfect separation of fraud versus legitimate transactions (AUC = 1.00, Recall = 1.00, and $F_1 = 0.99$). Although less sophisticated classifiers such as SVM, logistic regression, and QDA improved with oversampling, they remained well behind the ensemble methods in overall performance. TabNet, despite increasing its recall to around 0.87, still suffered from very low precision (≈ 0.23). Even in this balanced setting, XGBoost retained its speed advantage (≈ 3.8 seconds), while Random Forest and Extra Trees required tens to hundreds of seconds to train.

4.2.2 Performance on the 2019 Synthetic Dataset

In Table 5 the models are trained on the original dataset without oversampling. The results show that XGBoost and CatBoost attained high AUCs (0.99), but their recall lingered at about 0.57–0.58 and precision around 0.80–0.82, yielding F1 scores

Table 2 Performance metrics for the 2013 dataset without oversampling

Model	Accuracy	AUC	Recall	Precision	F1	Kappa	MCC	TT (s)
ET	0.99	0.93	0.77	0.96	0.85	0.85	0.86	8.57
CatBoost	0.99	0.97	0.78	0.95	0.86	0.86	0.86	35.27
RF	0.99	0.93	0.77	0.93	0.84	0.84	0.84	70.38
XGBoost	0.99	0.97	0.77	0.94	0.84	0.84	0.85	1.48
KNN	0.99	0.90	0.73	0.92	0.81	0.81	0.82	13.77
LDA	0.99	0.97	0.73	0.86	0.79	0.79	0.79	0.28
LR	0.99	0.97	0.61	0.87	0.72	0.72	0.73	0.84
AdaBoost	0.99	0.96	0.67	0.81	0.73	0.73	0.74	18.41
SVM	0.99	0.98	0.56	0.84	0.67	0.67	0.68	0.26
DT	0.99	0.86	0.72	0.74	0.73	0.73	0.73	6.71
Ridge	0.99	0.97	0.44	0.84	0.57	0.57	0.60	0.16
GBC	0.99	0.57	0.42	0.76	0.52	0.52	0.55	102.57
LightGBM	0.99	0.69	0.51	0.20	0.28	0.28	0.31	40.20
NB	0.97	0.95	0.82	0.06	0.11	0.11	0.22	0.17
QDA	0.97	0.96	0.86	0.05	0.10	0.10	0.21	0.31
TabNet	0.99	0.97	0.00	0.00	0.00	0.00	0.00	343.17

Table 3 Performance metrics for the 2013 dataset oversampling with SMOTE

Model	Accuracy	AUC	Recall	Precision	F1	Kappa	MCC	TT (s)
RF	0.99	1.00	1.00	0.99	0.99	0.99	0.99	120.70
ET	0.99	1.00	1.00	0.99	0.99	0.99	0.99	18.60
XGBoost	0.99	1.00	1.00	0.99	0.99	0.99	0.99	3.76
CatBoost	0.99	1.00	1.00	0.99	0.99	0.99	0.99	52.20
LightGBM	0.99	0.99	0.99	0.99	0.99	0.99	0.99	48.82
KNN	0.99	0.99	1.00	0.99	0.99	0.99	0.99	53.69
DT	0.99	0.99	0.99	0.99	0.99	0.99	0.99	11.35
GBC	0.97	0.99	0.96	0.98	0.97	0.95	0.95	212.84
AdaBoost	0.96	0.99	0.94	0.97	0.96	0.92	0.92	40.05
SVM	0.94	0.98	0.91	0.97	0.94	0.89	0.89	0.52
LR	0.94	0.98	0.91	0.97	0.94	0.89	0.89	1.48
QDA	0.92	0.98	0.88	0.96	0.92	0.85	0.86	0.90
Ridge	0.91	0.97	0.84	0.98	0.91	0.83	0.84	0.27
LDA	0.91	0.97	0.84	0.98	0.91	0.83	0.84	0.76
NB	0.91	0.94	0.85	0.97	0.90	0.82	0.83	0.27
TabNet	0.99	0.97	0.87	0.23	0.36	0.36	0.44	329.87

Table 4 Performance metrics for the 2013 dataset oversampling with ADASYN

Model	Accuracy	AUC	Recall	Precision	F1	Kappa	MCC	TT (s)
RF	0.99	1.00	1.00	0.99	0.99	0.99	0.99	141.97
ET	0.99	1.00	1.00	0.99	0.99	0.99	0.99	22.47
XGBoost	0.99	1.00	1.00	0.99	0.99	0.99	0.99	3.81
CatBoost	0.99	1.00	1.00	0.99	0.99	0.99	0.99	53.98
KNN	0.99	0.99	1.00	0.99	0.99	0.99	0.99	55.76
DT	0.99	0.99	0.99	0.99	0.99	0.99	0.99	15.46
LightGBM	0.99	0.99	0.99	0.99	0.99	0.99	0.99	48.97
GBC	0.97	0.99	0.98	0.96	0.97	0.94	0.94	213.23
AdaBoost	0.93	0.98	0.93	0.93	0.93	0.86	0.86	40.30
SVM	0.88	0.95	0.87	0.89	0.88	0.77	0.77	0.55
LR	0.88	0.95	0.86	0.90	0.88	0.76	0.77	1.46
QDA	0.86	0.95	0.76	0.95	0.85	0.73	0.74	0.75
Ridge	0.84	0.92	0.79	0.88	0.83	0.69	0.69	0.28
LDA	0.84	0.92	0.79	0.88	0.83	0.69	0.69	0.76
NB	0.71	0.84	0.47	0.91	0.62	0.43	0.49	0.28
TabNet	0.99	0.97	0.87	0.23	0.36	0.36	0.44	329.87

between 0.66 and 0.68. Many classifiers, such as logistic regression, ridge, and TabNet, failed to detect any fraud, often predicting all transactions as legitimate, whereas QDA produced an excessive number of false positives. In this scenario, XGBoost was again the fastest, training in 2.55 seconds compared to Random Forest's 27.8 seconds and CatBoost's 45.9 seconds.

When SMOTE or ADASYN was applied as shown in Tables 6 and 7, the top-performing models saw their recall climb to roughly 0.76-0.78, but precision dropped to around 0.50-0.54, leaving overall F1 scores near 0.62-0.63. Extra Trees and Random Forest struck a more favourable balance, achieving recalls of approximately

0.65–0.67 and precisions of about 0.70–0.73, which translated to F1 scores around 0.68. TabNet finally began to identify fraudulent transactions (recall up to 0.95 under ADASYN) but at the cost of producing an overwhelming number of false positives (precision ≈ 0.04), resulting in a very low F1 (≈ 0.07). Across these experiments, XGBoost consistently trained the fastest (6–7 seconds), while CatBoost (≈ 84 seconds) and TabNet (> 500 seconds) were substantially slower.

4.2.3 Performance Metrics for the 2023 Dataset

Remarkably, as shown in Table 8, without any oversampling, all leading ensemble models (Random Forest, Extra Trees, XGBoost, CatBoost) perfectly discriminated between fraudulent and non-fraudulent transactions (AUC = 1.00, recall = 1.00, F1 = 0.99). Simpler classifiers such as decision trees, AdaBoost, logistic regression, and QDA also performed robustly, with recall rates of at least 0.91, precision above 0.97, and F1 scores exceeding 0.94. For the first time, TabNet matched this high level of performance (AUC = 1.00, recall = 0.99, F1 = 0.99), although its training time increased to 717 seconds, substantially longer than XGBoost's 3.9 seconds or Random Forest's 134 seconds.

4.2.4 Inter Table Analysis

Ensemble techniques, especially XGBoost, Random Forest, and CatBoost, proved most effective across all datasets, consistently balancing high detection rates with reasonable training times. In the highly skewed 2013 data, these methods identified around 77–78% of fraud cases without oversampling and achieved perfect separation once SMOTE or ADASYN was applied, whereas TabNet failed and simpler models either overlooked fraud or generated excessive false alarms. With the synthetic 2019 data, oversampling boosted XGBoost's and CatBoost's recall to about

Table 5 Performance metrics for the 2019 dataset without oversampling

Model	Accuracy	AUC	Recall	Precision	F1	Kappa	MCC	TT (s)
RF	0.99	0.95	0.58	0.82	0.68	0.68	0.69	27.77
ET	0.99	0.94	0.53	0.84	0.65	0.64	0.66	13.45
XGBoost	0.99	0.99	0.57	0.80	0.66	0.66	0.67	2.55
CatBoost	0.99	0.99	0.58	0.79	0.67	0.66	0.67	45.92
GBC	0.99	0.72	0.34	0.73	0.45	0.45	0.49	29.21
DT	0.99	0.79	0.59	0.57	0.58	0.58	0.58	1.58
KNN	0.99	0.83	0.27	0.50	0.35	0.35	0.37	53.34
Ridge	0.99	0.83	0.00	0.00	0.00	0.00	0.00	0.58
AdaBoost	0.99	0.97	0.19	0.43	0.26	0.26	0.29	6.98
LR	0.99	0.79	0.00	0.00	0.00	0.00	0.00	3.21
SVM	0.99	0.49	0.01	0.17	0.01	0.01	0.03	1.81
LightGBM	0.99	0.86	0.46	0.43	0.44	0.44	0.44	36.18
NB	0.99	0.83	0.47	0.25	0.33	0.32	0.34	0.66
LDA	0.99	0.83	0.48	0.23	0.31	0.31	0.33	0.82
QDA	0.43	0.54	0.59	0.01	0.00	0.00	0.00	1.23
TabNet	0.99	0.92	0.00	0.00	0.00	0.00	0.00	291.31

Table 6 Performance metrics for the 2019 dataset oversampling with SMOTE

Model	Accuracy	AUC	Recall	Precision	F1	Kappa	MCC	TT (s)
ET	0.99	0.96	0.65	0.73	0.69	0.69	0.69	47.85
RF	0.99	0.96	0.66	0.71	0.68	0.68	0.68	64.93
DT	0.99	0.82	0.64	0.55	0.59	0.59	0.59	4.36
XGBoost	0.99	0.99	0.76	0.54	0.63	0.63	0.64	6.15
CatBoost	0.99	0.99	0.76	0.53	0.62	0.62	0.63	84.08
LightGBM	0.99	0.99	0.80	0.41	0.54	0.54	0.57	52.07
GBC	0.98	0.98	0.79	0.21	0.33	0.33	0.40	83.26
NB	0.97	0.81	0.70	0.10	0.18	0.17	0.26	1.13
AdaBoost	0.97	0.95	0.76	0.11	0.19	0.18	0.28	18.59
KNN	0.96	0.87	0.73	0.08	0.15	0.14	0.24	102.58
LR	0.91	0.88	0.71	0.03	0.06	0.05	0.14	23.04
Ridge	0.89	0.88	0.70	0.02	0.04	0.04	0.11	1.10
LDA	0.89	0.88	0.70	0.02	0.04	0.04	0.11	1.53
SVM	0.73	0.74	0.70	0.01	0.02	0.02	0.07	16.82
QDA	0.48	0.50	0.50	0.00	0.01	0.00	0.00	1.43
TabNet	0.97	0.99	0.92	0.15	0.25	0.25	0.36	1182.00

Table 7 Performance metrics for the 2019 dataset oversampling with ADASYN

Model	Accuracy	AUC	Recall	Precision	F1	Kappa	MCC	TT (s)
ET	0.99	0.96	0.65	0.72	0.68	0.68	0.68	38.88
RF	0.99	0.97	0.67	0.70	0.68	0.68	0.68	57.50
DT	0.99	0.82	0.65	0.55	0.60	0.59	0.60	5.07
CatBoost	0.99	0.99	0.77	0.52	0.62	0.62	0.63	84.28
XGBoost	0.99	0.99	0.78	0.51	0.61	0.61	0.63	6.82
LightGBM	0.99	0.99	0.83	0.36	0.51	0.50	0.55	50.16
GBC	0.98	0.98	0.82	0.18	0.29	0.29	0.38	79.71
NB	0.97	0.82	0.72	0.09	0.17	0.16	0.26	1.99
AdaBoost	0.96	0.95	0.78	0.08	0.15	0.14	0.25	18.82
KNN	0.96	0.87	0.73	0.07	0.13	0.13	0.22	102.57
Ridge	0.86	0.89	0.73	0.02	0.03	0.03	0.10	2.01
LDA	0.86	0.89	0.74	0.02	0.03	0.03	0.10	2.40
LR	0.85	0.89	0.74	0.01	0.03	0.03	0.10	23.11
SVM	0.68	0.75	0.72	0.01	0.02	0.01	0.07	16.43
QDA	0.43	0.59	0.66	0.00	0.00	0.00	0.01	2.40
TabNet	0.91	0.98	0.95	0.04	0.07	0.07	0.18	562.86

76–78%, but halved precision; Random Forest and Extra Trees offered a more balanced performance (recall $\approx$ 65–67%, precision $\approx$ 70–73%). By contrast, the 2023 dataset, characterised by less severe imbalance and richer features, allowed all major ensembles to attain near-perfect detection without any oversampling, and even straightforward classifiers (decision trees, AdaBoost, logistic regression, QDA) achieved recall $\geq$ 91% and $F_1 \geq$ 94%. TabNet succeeded only on the 2023 data, matching ensemble accuracy but requiring substantially more training time. Overall, these findings underscore that (1) ensembles deliver the best accuracy–efficiency trade-off under varied imbalance conditions; (2) oversampling is vital when fraud

Table 8 Performance metrics on 2023 dataset

Model	Accuracy	AUC	Recall	Precision	F1	Kappa	MCC	TT (s)
RF	0.99	1.00	1.00	0.99	0.99	0.99	0.99	133.74
ET	0.99	1.00	1.00	0.99	0.99	0.99	0.99	20.07
XGBoost	0.99	1.00	1.00	0.99	0.99	0.99	0.99	3.91
CatBoost	0.99	1.00	1.00	0.99	0.99	0.99	0.99	53.15
LightGBM	0.99	0.99	0.99	0.99	0.99	0.99	0.99	47.08
DT	0.99	0.99	0.99	0.99	0.99	0.99	0.99	13.34
GBC	0.97	0.99	0.96	0.98	0.97	0.95	0.95	221.44
AdaBoost	0.96	0.99	0.95	0.97	0.96	0.93	0.93	41.66
LR	0.96	0.99	0.95	0.97	0.96	0.92	0.92	14.61
QDA	0.95	0.98	0.91	0.98	0.94	0.90	0.90	1.17
Ridge	0.94	0.98	0.91	0.98	0.94	0.89	0.89	0.28
LDA	0.94	0.98	0.91	0.98	0.94	0.89	0.89	0.78
KNN	0.93	0.96	0.89	0.97	0.93	0.86	0.86	55.16
NB	0.91	0.97	0.84	0.98	0.90	0.83	0.83	0.29
SVM	0.85	0.96	0.85	0.89	0.85	0.70	0.73	12.42
TabNet	0.99	1.00	0.99	0.99	0.99	0.99	0.99	717.10

is exceedingly rare but can introduce false positives if synthetic examples are not realistic; and (3) as class imbalance recedes and features become more informative, simpler models become competitive alternatives.

As shown in Fig. 4, there are five bar charts that depict the mean values of key evaluation metrics: accuracy, AUC, recall, precision, and F1 score, across different tables. These charts offer a straightforward comparison of the models' overall effectiveness on these critical metrics, complementing the radar chart analysis. Additionally, Fig. 5, the radar chart, compares evaluation metrics across different tables, including accuracy, AUC, recall, precision, F1 score, Kappa, MCC, and normalised execution time (TT). It shows the strengths and weaknesses of each model based on these metrics, giving a clear and easy-to-understand picture of how well each model performs.

Table 5 shows the highest accuracy of 0.99 among all results. However, the values for AUC, recall, precision, F1-score, kappa, and MCC are much lower. This suggests that the dataset's limited features may have caused class imbalance or overfitting. It also shows why accuracy alone shouldn't be used to judge a model's performance.

Table 8 enhances accuracy, AUC, recall, precision, F1, kappa, and MCC by up to 0.39%, 18.92%, 167.51%, 137.90%, 184.29%, 176.12%, and 166.71%, respectively. The noticeable improvements in recall, precision, F1, kappa, and MCC are mainly because their starting values were quite low. The second-best overall performance, after Table 8, is seen in Table 4. While Table 4 used ADASYN oversampling to fix the dataset's imbalance, Table 8 didn't need it since the data was already balanced. Table 4 is better when the goal is to reduce false positives (high recall), while Table 8 works best for detecting actual fraud cases (high precision).

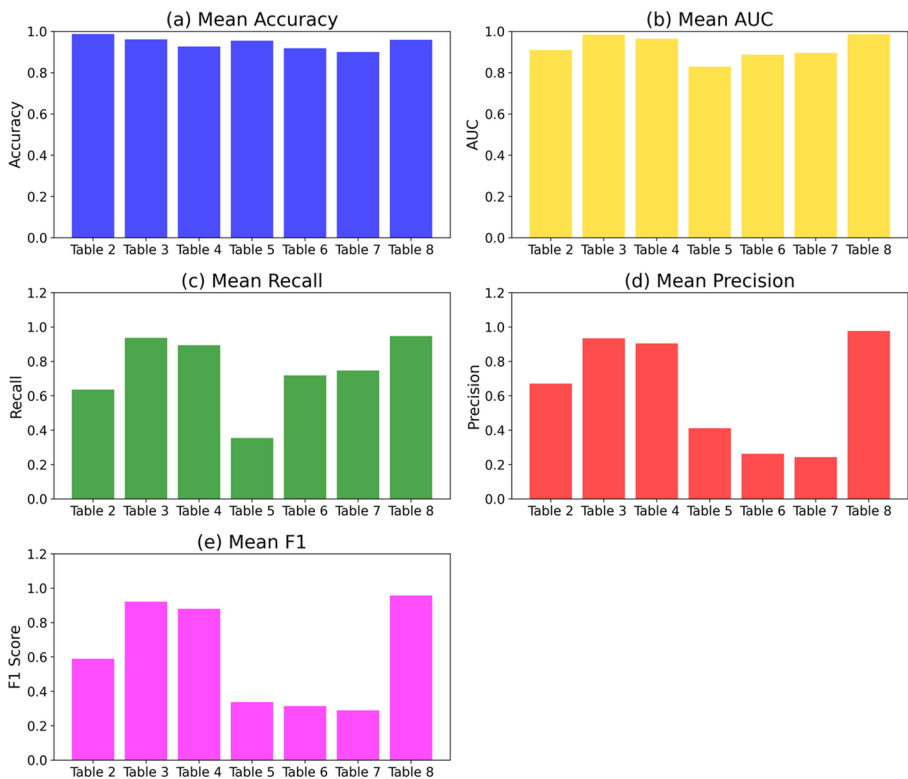


Fig. 4 Evaluation metrics across different tables, including (a) Mean Accuracy, (b) Mean AUC, (c) Mean Recall, (d) Mean Precision, and (e) Mean F1

4.3 SHAP-Based Model Interpretability

The following figures present SHAP summary plots for XGBoost (left panel) and CatBoost (right panel) models trained on the 2013, 2019, and 2023 datasets under different sampling conditions: no oversampling, SMOTE, and ADASYN. In each chart, the features are shown on the y-axis, ranked from most to least important based on their average SHAP values. Each horizontal section is a SHAP beeswarm plot, where every dot represents how much a single feature affected one prediction. The position on the x-axis shows how strongly and in what direction the feature influenced the result—positive values push the prediction toward “fraud”, while negative values push it toward “non-fraud”. The colour of each dot shows the actual value of that feature, with blue meaning low and red meaning high, helping to visualise how different feature levels affect the model’s decisions.

4.3.1 SHAP Analysis on 2013 Dataset

As shown in Fig. 6, SHAP analyses of XGBoost and CatBoost on the 2013 fraud dataset indicate that both algorithms identify transaction amount, V14 and V4 as the most significant predictors of fraudulent activity. However, XGBoost’s SHAP dis-

Radar Chart of Metrics by Table

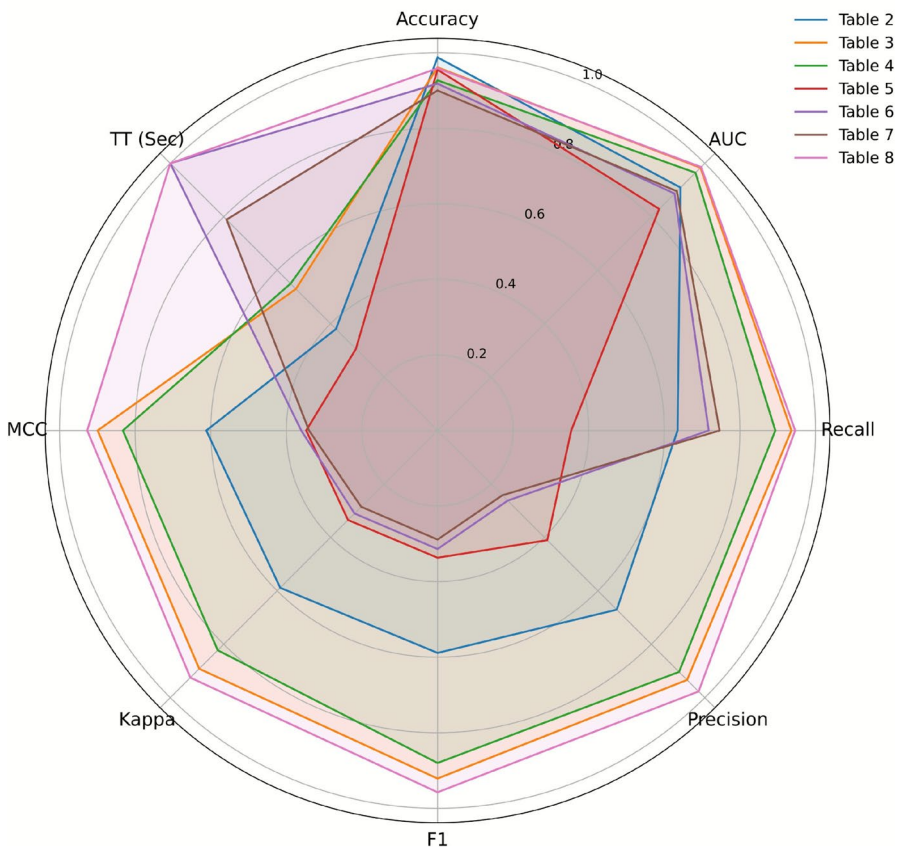


Fig. 5 Radar chart by metrics

tributions display larger spreads, suggesting a more pronounced reaction to extreme feature values, whereas CatBoost elevates the importance of V1, V2, and V8, thereby capturing nonlinear patterns that XGBoost underemphasises. As a result, XGBoost's classification boundary is more reactive to outliers, while CatBoost dampens the influence of extreme observations and draws attention to y features (V1, V2, V8) for potential feature engineering.

After SMOTE balancing as shown in Fig. 7, both XGBoost and CatBoost still regard V14 and V4 as their top predictors, but the next highest features shift: XGBoost ranks V12, V17, and V3 as fourth and fifth, while CatBoost places V12, V1, and V3 in those positions. V8 declines in importance for CatBoost, and transaction amount falls to last for both models, reflecting its loss of discriminatory power on synthetically generated data. SMOTE also narrows XGBoost's SHAP distributions, reducing its sensitivity to extreme values, and causes both algorithms to distribute importance more evenly across V12, V17, and V3, while preserving V14 and V4 as the primary fraud-detection signals.

Figure 8 shows that with the application of ADASYN oversampling, both XGBoost and CatBoost continue to highlight V14 and V4 as the most critical features for fraud detection. However, features such as V1, V8, and V12 become more significant compared to earlier sampling techniques. The contribution of the “Amount” feature remains relatively minor in both models. XGBoost exhibits wider SHAP value distributions, suggesting greater sensitivity to specific features, whereas CatBoost shows a more consistent impact across features. In essence, ADASYN strengthens the relevance of minority class indicators like V1 and V8 while consistently reaffirming the dominant role of V14 and V4.

4.3.2 SHAP Analysis of 2019 Dataset

Figure 9 compares how both models, XGBoost and CatBoost, decide which features matter most in the 2019 dataset without using any oversampling. Both the models show that the amount (amt) is the biggest clue for detecting fraud. After that, their focus changes. XGBoost pays more attention to numerical and location details, such as population, state, and merchant coordinates. CatBoost, however, pays more attention to the kind of transaction, for example, whether it was a home-related purchase or something bought in a store.

The SHAP plots also show that XGBoost reacts more strongly to very large values, while CatBoost spreads its attention more evenly across many features.

Figure 10 compares SHAP values from the XGBoost and CatBoost models applied to the 2019 dataset using SMOTE oversampling. In both models, the transaction amount (amt) is the most important feature. XGBoost focuses more on numerical

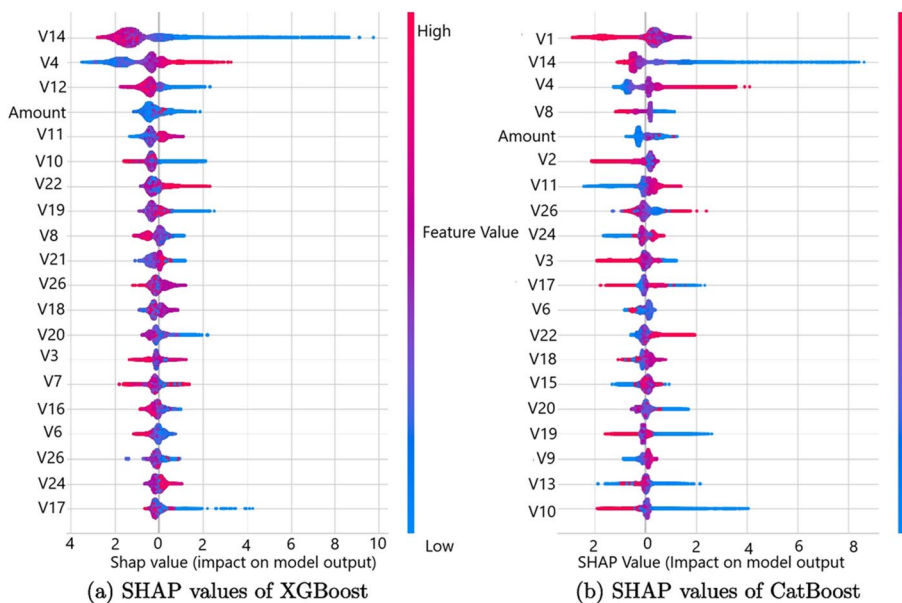


Fig. 6 Comparison of SHAP values between XGBoost and CatBoost without oversampling on 2013 dataset

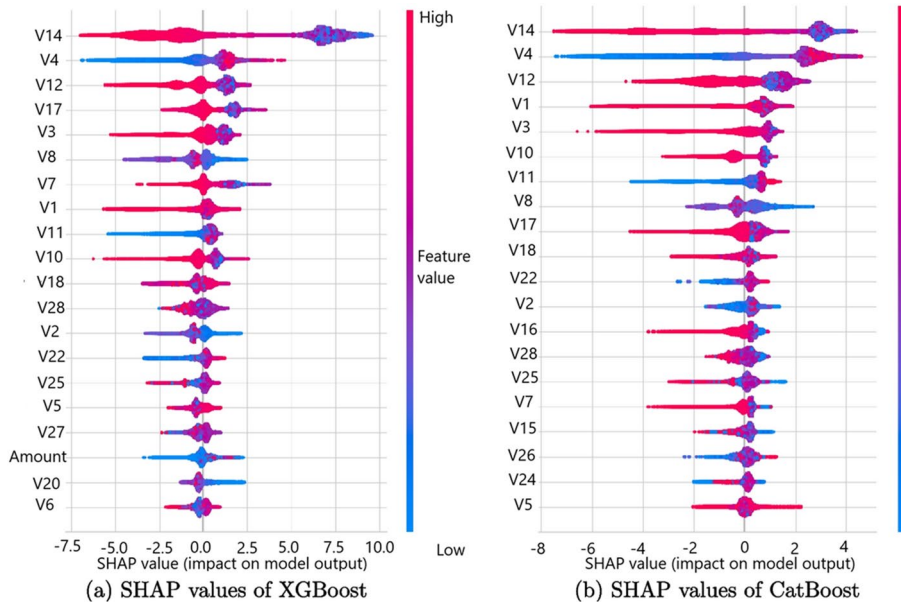


Fig. 7 Comparison of SHAP values between XGBoost and CatBoost oversampling with SMOTE on 2013 dataset

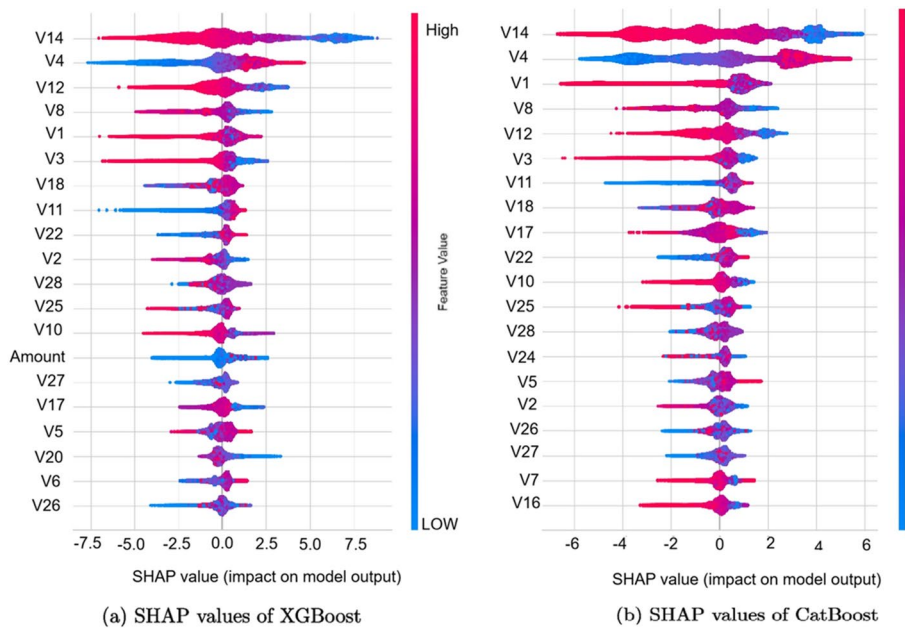


Fig. 8 Comparison of SHAP values between XGBoost and CatBoost oversampling with ADASYN on 2013 dataset

and location-based features like `city_pop`, `state`, and `merch_lat`, while CatBoost places greater importance on categorical features such as `category_shopping_pos`, `category_entertainment`, and `category_home`. SMOTE helps create a more balanced spread of feature importance in both models. Overall, XGBoost tends to rely more on numeric information, whereas CatBoost is better at capturing patterns in categorical data, showing how each algorithm views the fraud detection problem differently.

Figure 11 illustrates how XGBoost and CatBoost interpret fraud detection differently when trained on the 2019 dataset with ADASYN oversampling. Both models identify the transaction amount (`amt`) as the most important feature. However, XGBoost relies more on numerical and location-based details, such as city population and state, while CatBoost focuses more on the type of transaction, like shopping or home-related expenses. ADASYN helps in balancing the dataset by oversampling, and it allows both models to look at a wider variety of features instead of relying on the few features.

4.3.3 SHAP Analysis of 2023 Dataset

Figure 12 shows that both XGBoost and CatBoost agree on the top features for spotting fraud in the 2023 dataset, mainly V14, V4, V12, and V3. XGBoost spreads its attention across many different features, picking up on a wide range of patterns, even the small or complicated ones. CatBoost, however, focuses more on a smaller group of very important features, giving a clearer but narrower view. In simple terms, XGBoost looks broadly across the data, while CatBoost concentrates more deeply on just a few key signals.

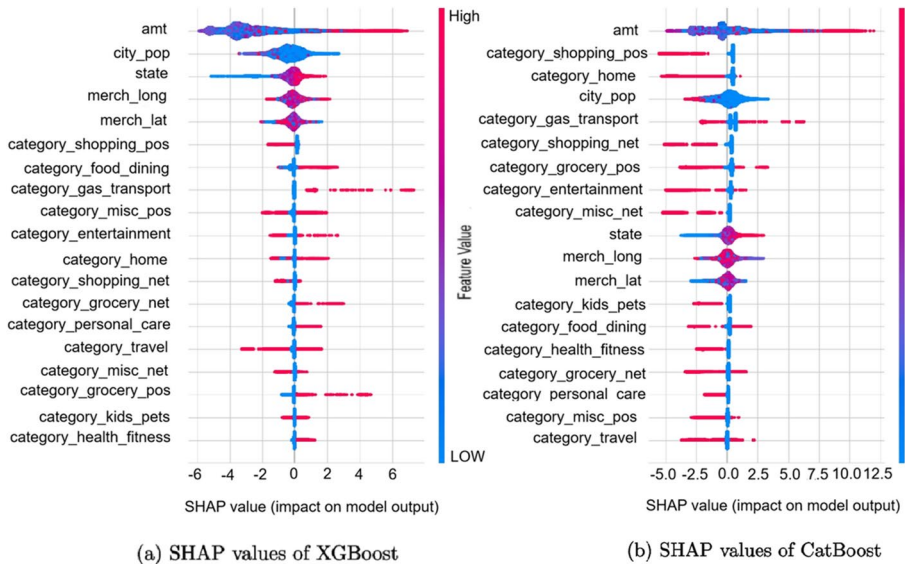


Fig. 9 Comparison of SHAP values between XGBoost and CatBoost without oversampling on 2019 dataset

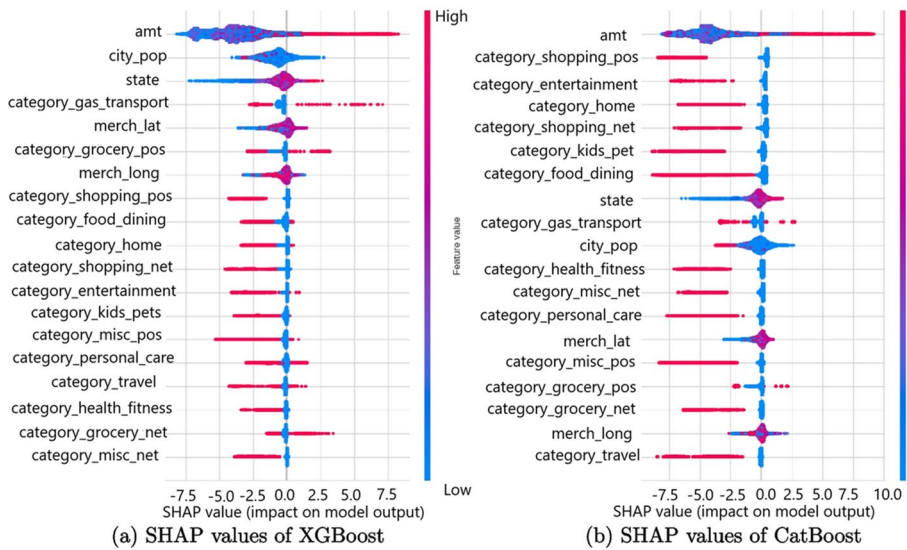


Fig. 10 Comparison of SHAP values between XGBoost and CatBoost oversampling with SMOTE on 2019 dataset

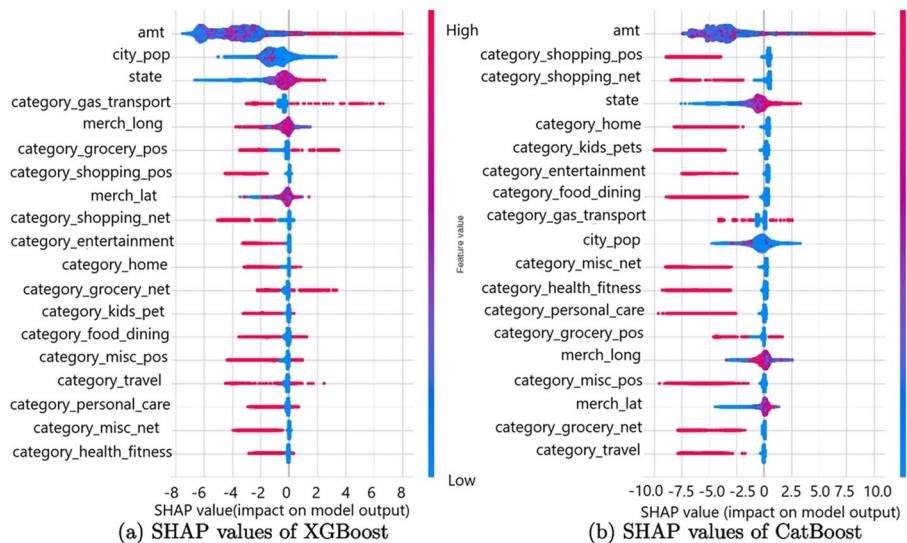


Fig. 11 Comparison of SHAP values between XGBoost and CatBoost oversampling with ADASYN on 2019 dataset

4.4 Challenges in Credit Card Fraud Detection

Detecting credit card fraud is hard for several reasons. First, fraud doesn't happen very often less than 1% of all the transactions so most of the data is normal transactions. Second, criminals keep changing their way of making fraud, so fraud detection

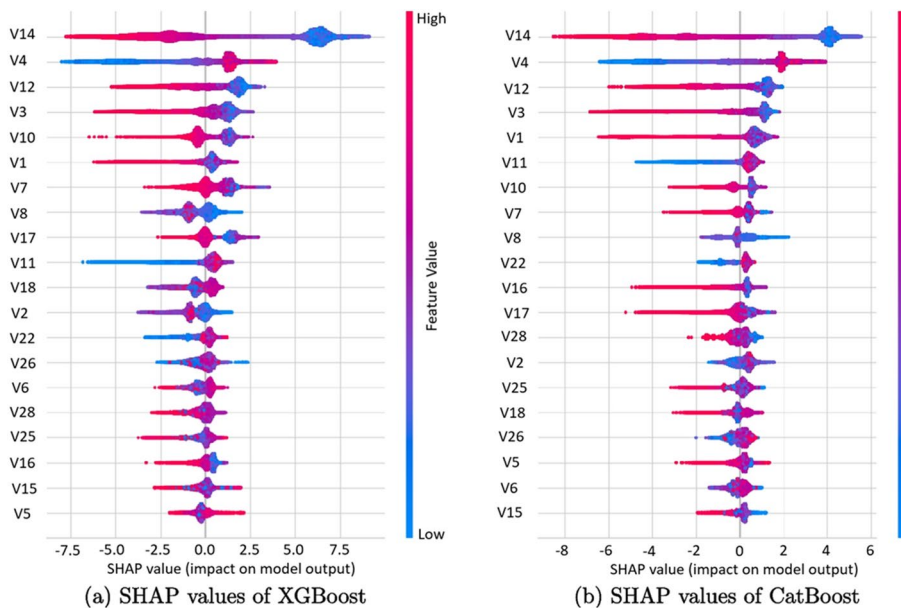


Fig. 12 Comparison of SHAP values between XGBoost and CatBoost on 2023 dataset

systems have to keep learning and need to be updated to stay effective. Another big issue is speed. Fraud checks need to happen in real time so customers do not have to wait during their transactions. Lastly, there are strict privacy rules, like GDPR, that protect people's personal information. Fraud detection systems should ensure to follow these rules and the data safe.

4.5 Limitations

The study presents a comprehensive evaluation of the ML model, however there are some limitations. Firstly, there are very few fraud incidents in the dataset, hence performance can look good even when there are a lot of frauds cases are missed. Oversampling might create some artificial or noisy patterns. The models were only trained once, so they would not learn new patterns of fraud methods that arise up over time.

5 Conclusion and Future Scope

In this study, authors tested 15 different ML models to detect credit card fraud. It has been found that XGBoost and CatBoost work best for detection fraud when most transactions are normal, while TabNet does well on the balanced dataset but needs more computing power. By using the SHAP, we could see the influencing the model's decisions. For real- world use, it's best to use ensemble models, keep updating them as fraud changes, and use tools that explain why a model flags a transaction. Future research should look into more advanced ML models, like Graph-based neural networks and transformer. Methods like reinforcement learning and sequential decision

making models could help build systems that adapt to fraudsters as they changes their way of crime. For fraud detection systems to work in real-time, systems need to be fast, scalable and need of explainable AI methods like SHAP or LIME.

Acknowledgements We also thank the original researchers and participants whose work and support made this study possible.

Declarations

Ethical Compliance The study was conducted in accordance with all ethical rules and guidelines of our institution. Care was taken to handle the data responsibly and ethically.

Conflicts of Interest The authors declare no potential conflict of interest.

References

- Ali, M. (2020) *PyCaret: An open source, low-code machine learning library in Python*. <https://www.pycaret.org>. PyCaret version 1.0.0.
- Arik, S. O., & Pfister, T. (2021). Tabnet: Attentive interpretable tabular learning. In *Proceedings of the AAAI conference on artificial intelligence* (Vol. 35, pp. 6679–6687).
- Awoyemi, J. O., Adetunmbi, A. O., & Oluwadare, S. A. (2017) Credit card fraud detection using machine learning techniques: A comparative analysis. In *2017 International Conference on Computing Networking and Informatics (ICCN)* (pp. 1–9). IEEE
- Bolton, R. J., & Hand, D. J. (2002). Statistical fraud detection: A review. *Statistical Science*, 17(3), 235–255.
- Breiman, L. (2017). *Classification and regression trees*. Routledge.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5–32.
- Chawla, N. V., Bowyer, K. W., & Hall, L. O. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- Chen, T. & Guestrin, C. (2016) XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 785–794).
- Chhabra, R., Goswami, S., & Ranjan, R. K. (2024). A voting ensemble machine learning based credit card fraud detection using highly imbalance data. *Multimedia Tools and Applications*, 83(18), 54729–54753.
- Cortes, C. (1995). Support-vector networks. *Machine Learning*.
- Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 20(2), 215–232.
- Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. In *2015 IEEE symposium series on computational intelligence* (pp. 159–166). IEEE
- Domingo, C., Watanabe, O., et al. (2000). MadaBoost: A modification of AdaBoost. In *Colt* (pp. 180–189).
- Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188.
- Fix, E. (1985) *Discriminatory analysis: Nonparametric discrimination, consistency properties*, vol. 1. USAF school of Aviation Medicine.
- Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119–139.
- Friedman, J. H. (2001) Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, pp. 1189–1232.
- Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely randomized trees. *Machine Learning*, 63, 3–42.
- Hafez, I. Y., Hafez, A. Y., Saleh, A., Abd El-Mageed, A. A., & Abohany, A. A. (2025). A systematic review of AI-enhanced techniques in credit card fraud detection. *Journal of Big Data*, 12(1), 6.

- He, H., Bai, Y., Garcia, E. A., & Li, S. (2008) ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence)* (pp. 1322–1328). IEEE.
- Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.
- Kartik2112 (2021) Fraud detection. <https://www.kaggle.com/datasets/kartik2112/fraud-detection>. Accessed 16 Nov 2024.
- Khalid, A. R., Owoh, N., Uthmani, O., Ashawa, M., Osamor, J., & Adejoh, J. (2024b) Enhancing credit card fraud detection: An ensemble machine learning approach. *Big Data and Cognitive Computing*, 8 (1). <https://doi.org/10.3390/bdcc8010006>, <https://www.mdpi.com/2504-2289/8/1/6>. ISSN 2504-2289
- Khalid, A. R., Owoh, N., Uthmani, O., Ashawa, M., Osamor, J., & Adejoh, J. (2024a). Enhancing credit card fraud detection: An ensemble machine learning approach. *Big Data and Cognitive Computing*, 8(1), 6.
- Khalil, A. A., Liu, Z., Salah, A., & Fathalla, A. (2022). Predicting insolvency of insurance companies in Egyptian market using bagging and boosting ensemble techniques. *IEEE Access*, 10, 117304–117314.
- Khalil, A. A., Liu, Z., Fathalla, A., Ali, A., & Salah, A. (2024). Machine learning based method for insurance fraud detection on class imbalance datasets with missing values. *IEEE Access*, 12, 155451–155468. <https://doi.org/10.1109/ACCESS.2024.3468993>
- Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., & Willing, C. (2016) Jupyter notebooks – a publishing format for reproducible computational workflows. In F. Loizides, & B. Schmidt (Eds.) *Positioning and power in academic publishing: Players, agents and agendas* (pp. 87 – 90). IOS Press.
- Kohavi, R. (1998). Glossary of terms. *Machine Learning*, 30, 271–274.
- Manghi, M., Palmeri, J., Yazda, K., Henn, F., & Jourdain, V. (2018). Role of charge regulation and flow slip in the ionic conductance of nanopores: An analytical approach. *Physical Review E*, 98(1), 012605.
- Nelgiryewithana (2023). Credit card fraud detection dataset 2023. <https://www.kaggle.com/datasets/nelgiryewithana/credit-card-fraud-detection-dataset-2023/discussion/459020>. Accessed 26 Nov 2024.
- Nilson, R. (2023). Nilson report: Newsletter 1254. <https://nilsonreport.com/newsletters/1254/>. Accessed 23 Nov 2024.
- OpenML Contributors (2025). Openml dataset 1597. <https://www.openml.org/d/1597>. Accessed 12 June 2025.
- Patidar, R., Sharma, L., et al. (2011). Credit card fraud detection using neural network. *International Journal of Soft Computing and Engineering (IJSCE)*, 1 (32–38).
- Pérez, & Granger, B. E. (2007). IPython: A system for interactive scientific computing. *Computing in Science and Engineering*, 9(3): 21–29. <https://doi.org/10.1109/MCSE.2007.53>, <https://ipython.org>. ISSN 1521-9615.
- Phua, C., Lee, V., Smith, K., & Gayler, R. (2010) A comprehensive survey of data mining-based fraud detection research. [arXiv:1009.6119](https://arxiv.org/abs/1009.6119).
- Potluri, P., Suneel, S., Chandra, B., Muthukumaran, D., Upadhyaya, M., & Kanna, R. R. (2024). ELCCFD: An efficient and enhanced credit card fraud detection using enhanced deep learning principle. In *2024 5th International Conference on Image Processing and Capsule Networks (ICIPCN)* (pp. 337–343). IEEE.
- Powers, D. M. W. (2020). Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. [arXiv:2010.16061](https://arxiv.org/abs/2010.16061).
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018) CatBoost: Unbiased boosting with categorical features. *Advances in neural information processing systems*, 31.
- Python Software Foundation (2021). *Python Language Reference, version 3.10*. <https://www.python.org/>.
- Randhawa, K., Loo, C. K., Seera, M., Lim, C. P., & Nandi, A. K. (2018). Credit card fraud detection using Adaboost and majority voting. *IEEE Access*, 6, 14277–14284.
- Rao, C. R. (1948). The utilization of multiple measurements in problems of biological classification. *Journal of the Royal Statistical Society. Series B (Methodological)*, 10(2), 159–203.
- Rish, I. et al. (2001). An empirical study of the naive Bayes classifier. In *IJCAI 2001 workshop on empirical methods in artificial intelligence*. Seattle, WA, USA.
- Van Rossum, G., & Drake, Jr. F. L. (1995). *Python tutorial*, vol. 620. Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands.

- Zareapoor, M., Shamsolmoali, P., et al. (2015). Application of credit card fraud detection: Based on bagging ensemble classifier. *Procedia Computer Science*, 48(2015), 679–685.
- Zhu, M., Zhang, Y., Gong, Y., Xu, C., Xiang, Y. (2024) Enhancing credit card fraud detection a neural network and SMOTE integrated approach. [arXiv:2405.00026](https://arxiv.org/abs/2405.00026).

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Deepankur Singh¹ · Simranjit Singh¹  · Neeraj Kumar¹

✉ Simranjit Singh
simran.jita@gmail.com

Deepankur Singh
deepankur1singh@gmail.com

Neeraj Kumar
kumarneeraj@nitj.ac.in

¹ Department of Information Technology, Dr BR Ambedkar National Institute of Technology Jalandhar, Punjab, India